

The Autopoietic Codex: Architecting Self-Improving Agentic Systems within the Google Antigravity Ecosystem

1. Introduction: The Agentic Phase Transition

The trajectory of software engineering is currently navigating a profound phase transition, evolving from the era of "copilots"—where artificial intelligence acts as a subservient suggestion engine within a static integrated development environment (IDE)—to the era of "agentic orchestration." In this emerging paradigm, the developer transitions from a typist of syntax to an architect of cognition, managing autonomous digital entities capable of planning, execution, and verification.¹ The release of Google Antigravity, an agent-first development platform, marks the crystallization of this shift, decoupling the "mission control" of agent management from the "implementation surface" of code editing.²

This report serves as a definitive technical blueprint for constructing a highly specialized, self-improving agent within the Antigravity ecosystem. The objective is to engineer an entity that transcends static programming to achieve recursive domain mastery. This agent is designed not merely to execute tasks but to autonomously acquire the documentation, context, and skills necessary to navigate a hierarchy of ascending complexity: from the foundational substrate of **Programming**, through the information acquisition of **Research**, into the probabilistic systems of **Finance**, the rigid logic of **Law**, and finally, the meta-systemic dynamics of **Politics**.

Crucially, this architecture addresses the fundamental cognitive limitation of Large Language Models (LLMs)—the finite context window. We define a rigorous implementation of the "Breathe-In, Breathe-Out" protocol using the Gemini CLI, ensuring the agent can autonomously compress its memory into high-fidelity artifacts and perform a "tabula rasa" reset to maintain infinite operational continuity without context degradation.⁴ Furthermore, we solve the "documentation fragmentation" problem inherent in bleeding-edge tools by equipping the agent with autonomous crawling and synthesis capabilities, allowing it to generate its own instruction manuals from scattered web sources.⁷

The following analysis synthesizes technical documentation, API specifications, and architectural patterns to provide the "next Gemini 3 Pro instance" with the wealth of information required to instantiate this autopoietic system.

2. Part I: The Vessel — Architecting the Google Antigravity Environment

To build a self-improving agent, one must first master the physics of the environment it inhabits. Google Antigravity is not merely an editor; it is a bifurcated workspace designed to facilitate the interaction between human intent and machine execution. Understanding this dichotomy is the first step in enabling an agent to "teach itself."

2.1 The Dichotomy of Surfaces: Decoupling Cognition from Execution

The core innovation of Antigravity is the separation of "doing" from "managing," a distinction that resolves the cognitive dissonance of trying to vibe-code while simultaneously orchestrating complex, multi-step workflows.

2.1.1 The Editor View: The Synchronous Implementation Layer

The Editor View represents the traditional IDE experience, enhanced by models such as Gemini 3 Pro. It is a synchronous, low-latency environment optimized for "vibe coding"—the rapid, intuitive generation of code where the user and AI collaborate in real-time.¹ However, for a self-improving agent, the Editor is simply the *output* destination. It is the canvas upon which the agent paints, but it is not where the agent *thinks*. Relying solely on the Editor for autonomous tasks leads to "context switching" fatigue for the human and "context drift" for the AI, as the conversational history becomes cluttered with diffs and syntax corrections.¹

2.1.2 The Agent Manager: The Asynchronous Cognitive Core

The Agent Manager, or "Mission Control," is the locus of autonomy. This distinct interface allows the developer to spawn, orchestrate, and observe multiple agents working asynchronously across different workspaces.¹

- **Asynchronous Parallelism:** Agents in the Manager can run in the background. A "Research Agent" can be spawned to crawl documentation on legiscan.com for political data, while a "Finance Agent" backtests a trading strategy in the Editor. This parallelism is essential for the multi-domain requirement.³
- **Plan Mode vs. Fast Mode:** The Agent Manager supports distinct cognitive modes. **Fast Mode** executes instructions instantly, ideal for quick fixes. **Plan Mode** forces the agent to generate a detailed "Implementation Plan" artifact before writing a single line of code.¹¹ For a self-improving agent, Plan Mode is mandatory. It acts as a "reasoning buffer," allowing the agent to verify its approach against known best practices before committing resources to execution.
- **Artifact-Driven Communication:** Agents communicate via **Artifacts**—structured, persistent outputs like Task Lists, Implementation Plans, and Walkthroughs.¹ These Artifacts serve as *external memory*. By forcing the agent to write its state to an artifact, we persist its reasoning process independent of the volatile context window, a critical requirement for long-running tasks.

Strategic Implication: To build the requested "specialized agent," one must configure the Antigravity environment to prioritize the Agent Manager. The agent must be taught to utilize Artifacts not just as communication tools for the human user, but as a "journal" for its own self-correction loop.

2.2 Configuration for Autonomy: The settings.json Architecture

To enable an agent to "teach itself" to navigate the fragmented Antigravity documentation, we must configure its environment for maximum permission and minimal friction, while maintaining a safety sandbox. The default configuration of Antigravity is designed for safety ("Review-driven development"), requiring human approval for most actions.¹¹ A self-improving agent requires "Agent-driven development" or "Autopilot."

The settings.json file is the nervous system of the Antigravity setup. A standard installation provides safe defaults; a self-improving agent requires aggressive customization to enable the "OODA loop" (Observe, Orient, Decide, Act) of autonomous learning.

Table 1: Critical settings.json Parameters for Autonomous Agents

Setting Parameter	Recommended Value	Purpose & Implication	Source
agent.mode	"plan"	Enforces the generation of a "Plan Artifact" before execution. This creates a verification step where the agent can self-correct its logic against documentation before generating code.	²
admin.secureModeEnabled	false	Disables the "Human-in-the-Loop" block for low-risk actions. This allows the agent to execute shell commands (like installing new Python libraries or cloning git repos) without blocking for confirmation.	⁵
terminal.policy	"auto"	Critical. If set to manual, the agent halts for every pip install or git clone. Setting to auto allows the agent to encounter a "Module Not Found"	¹¹

		error, reason about it, and execute the fix autonomously.	
agent.workspace.allowExternalFileAccess	true (with AllowList)	By default, Antigravity sandboxes the agent to the current workspace. To allow the agent to learn from other projects or manage a central "Memory" repository, this must be carefully managed via the Allow List.	¹²
browser.headless	true	Enables background research. The agent can spawn a browser instance to crawl documentation without stealing focus from the user.	¹¹

Configuration Strategy for Autonomous Agents

Setting Key	 Default Profile (Safe)	 Autonomous Profile (Risk)
<code>agent.files.access</code> Defines file system scope	Workspace & ~/.antigravity	Non-Workspace Access (Read/Edit) 
<code>terminal.policy</code> Shell command permissions	Blocked / Restricted	Execute Terminal Commands 
<code>browser.security</code> Web navigation controls	Allowlist Only (Explicit Trust)	Denylist Only (Broad Access) 
<code>telemetry.enabled</code> Data collection for improvement	False (Default)	True 

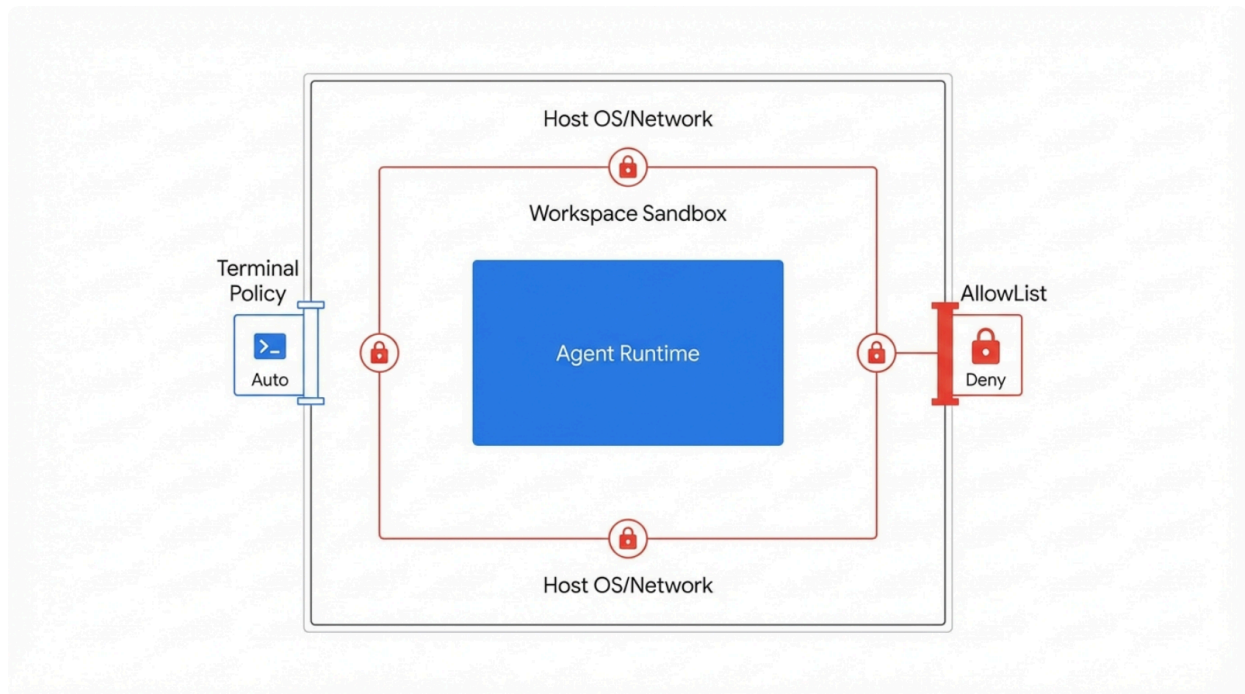
To enable self-improvement, the agent requires elevated permissions. The 'Autonomous' profile grants the agent the ability to execute terminal commands and access external files, which is necessary for research and self-modification but increases security risk.

Data sources: [Antigravity Modes](#), [Settings Reference](#), [Security Allowlist](#)

2.2.1 The Permissions Hierarchy

It is vital to visualize the security layers because opening up an agent for self-improvement inherently increases risk. We must balance the agent's need to access the OS (to install tools) with the need to protect the host system. The Antigravity security model relies on concentric rings of permission. The 'Agent' resides in the center, bounded by the Workspace. Access to the Terminal and Browser is mediated by Policy gates (Auto vs. Manual), while file system access is controlled by rigid Allow/Deny lists.²

Antigravity Security & Autonomy Architecture



The Antigravity security model relies on concentric rings of permission. The 'Agent' resides in the center, bounded by the Workspace. Access to the Terminal and Browser is mediated by Policy gates (Auto vs. Manual), while file system access is controlled by rigid Allow/Deny lists.

2.3 The "Skill" Construct: Modularizing Competence

The most powerful feature for self-improvement in Antigravity is the concept of **Skills**. Rather than hardcoding capabilities into a massive system prompt—which leads to "Tool Bloat" and context degradation—Skills allow for **Progressive Disclosure**.¹⁴

2.3.1 The Anatomy of a Skill

Skills are defined in SKILL.md files located in the `.agent/skills/` directory. They represent a specialized unit of capability. Crucially, the agent only sees the *metadata* of a skill (its description and triggers) in its active context. It loads the heavy procedural knowledge (instructions and scripts) only when the user's intent matches the skill.¹⁴

This architecture is the key to the "Self-Teaching" requirement. If the agent encounters a gap in its knowledge (e.g., "I do not know how to query the LegiScan API"), it does not need a prompt engineering update. Instead, it needs to:

1. **Research** the API documentation (using its Crawler skill).
2. **Generate** a new SKILL.md file encapsulating that API.
3. **Save** it to `.agent/skills/`.

By doing this, the agent effectively "installs" new knowledge into its own brain. The directory structure for a skill is standardized to ensure portability ¹⁵:

my-skill/

- |— SKILL.md # The definition file (Prompts & Triggers)
 - |— scripts/ # Python, Bash, or Node scripts for execution
 - | — run.py
 - |— references/ # Documentation or templates (e.g., API schemas)
 - |— assets/ # Static assets
-

3. Part II: The Mind — Cognitive Hygiene and the Gemini CLI

The user specifically requested a mechanism for the Gemini CLI to "compress its context window autonomously... and after creating the compressed extractive summary it can totally empty its context window." This is a sophisticated request that addresses the core limitation of LLMs: the Finite Context Problem. Even with Gemini 3 Pro's massive context window, performance degrades (latency increases, reasoning sharpness blunts) as the buffer fills. This phenomenon, often referred to as "Context Rot" ¹⁴, necessitates a rigorous "Garbage Collection" mechanism for cognitive load.

3.1 The "Compress & Flush" Lifecycle

To achieve the user's requirement of a "total emptying" while retaining continuity, we must engineer a lifecycle using Gemini CLI's advanced hook system (hooks.PreCompress, hooks.AfterModel) and specific model configuration settings.⁵ Standard compression (/compress) merely replaces the history with a summary *within* the context.⁴ The user demands a "Tabula Rasa" (clean slate) state, which requires a session handoff architecture.

Phase 1: The Trigger Configuration

We utilize the model.compressionThreshold setting to automate the detection of context saturation.

- **Setting:** "model.compressionThreshold": 0.4
- **Effect:** When the context usage hits 40% of the limit (e.g., 400k tokens of a 1M window), the compression sequence initiates automatically.⁵ Setting this aggressively low ensures the model never reaches the "degradation zone" where it begins to hallucinate or ignore instructions.⁶

Phase 2: The Extractive Summary (Pre-Compression)

Before the default compression wipes the history, we need to extract a high-fidelity summary. We cannot rely solely on the default summarizer; we want a structured "Memory Artifact" that persists on disk.

We define a hook in settings.json that triggers a custom Python script:

JSON

"hooks.PreCompress":

The extract_knowledge_graph.py script will:

1. Read the current session history from ~/.gemini/tmp/<project_hash>/chats/.¹⁶
2. Send a specific prompt to the model (via a side-channel API call): *"Synthesize the current session into a persistent Knowledge Object. Discard conversational filler. Retain only: Code Snippets, Key Decisions, Solved Errors, and Pending Tasks."*
3. Save this output to MEMORY.md in the project root. This file acts as the "Long-Term Memory" (LTM).

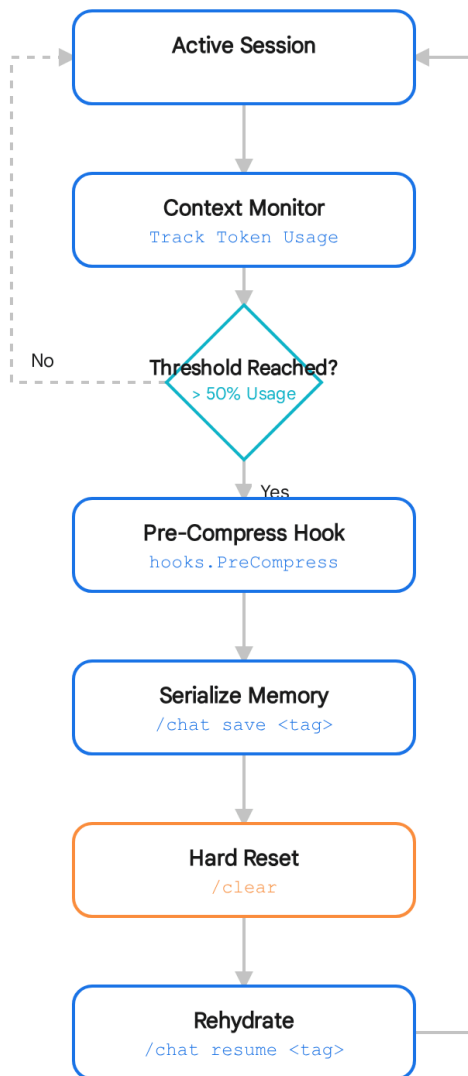
Phase 3: The Hard Flush (The "Total Emptying")

The user requested to "totally empty" the context window. Standard compression leaves a summary *in* the context. To achieve a true "Tabula Rasa" while retaining continuity, we must use a radical approach: **Session Handoff**.

Instead of just compressing, the agent will:

1. **Serialize** critical state to MEMORY.md (Phase 2).
2. **Execute** the /clear command. As per documentation, /clear provides a "hard reset" that "wipes the current conversational context".⁴ This reduces the token count to near zero.
3. **Rehydrate** by immediately injecting the MEMORY.md content as the new system prompt or the first user message. This ensures the agent "remembers" the mission but is unburdened by the thousands of tokens of previous trial-and-error.

Autonomous Context Compression & Flush Cycle



This cycle ensures the agent never suffers from 'context rot.' By offloading memory to persistent storage (MEMORY.md) before performing a hard reset, the agent maintains long-term continuity with short-term agility.

Data sources: [Medium \(Gemini CLI Tutorial\)](#), [Gemini CLI Docs \(Config\)](#), [Reddit \(r/GeminiAI\)](#).

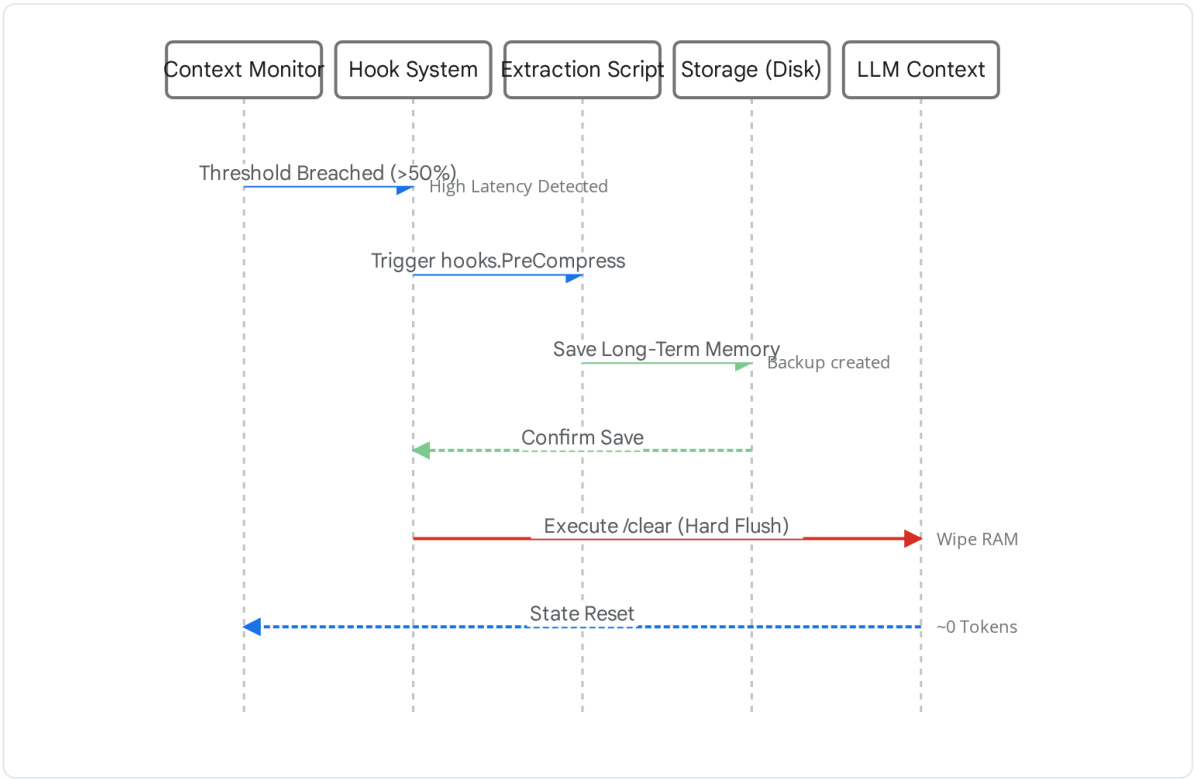
3.2 Implementing the Sequence

The specific request implies a state machine logic that is complex to describe in prose. The following sequence diagram clarifies the interactions between the User, the CLI, the LLM, and the File System during this automated cycle.

Autonomous Context Compression & Flush Sequence

System State Logic: Threshold → Extract → Flush

● System/Hook ● Storage I/O ● Critical Reset



This sequence demonstrates the 'Breathe-In, Breathe-Out' cycle. When the Token Threshold is breached, the system halts, extracts 'Long-Term Memory' to disk, and then performs a 'Hard Flush' (/clear) to restore the context window to near-zero latency.

Data sources: [Medium \(Google Cloud\)](#), [Gemini CLI Docs](#), [Reddit \(r/GeminiAI\)](#)

4. Part III: The Senses — Autonomous Research and Documentation

The user explicitly noted that Antigravity's documentation is fragmented ("click through segments... merge multiple files") and that the agent should be able to "teach himself." This is a universal problem for cutting-edge tools. A self-improving agent must be able to ingest raw, scattered web documentation and synthesize it into a coherent llms.txt or knowledge.md file.⁷

4.1 The Documentation Crawler: crawl4ai

To teach the agent to research, we must equip it with **Headless Browsing** and **Markdown Conversion** capabilities. We will integrate crawl4ai (Python) as the sensory input mechanism. Unlike standard scrapers (e.g., BeautifulSoup), crawl4ai uses a headless browser (Chromium) to render JavaScript-heavy documentation sites and outputs "LLM-ready" Markdown directly.⁸

The "Doc-Hunter" Agent Skill:

We create a skill skills/research/ingest_docs.md.

- **Input:** A base URL (e.g., <https://antigravity.google/docs>).
- **Configuration:** The agent uses the PruningContentFilter strategy. This is crucial because modern documentation sites are filled with "noise" (navbars, footers, "Edit on GitHub" links) that waste tokens. The pruning filter strips this, leaving only the semantic signal.²⁰
- **Workflow:**
 1. **Target Identification:** The agent identifies a documentation hub.
 2. **Recursive Crawl:** The agent executes a Python script utilizing AsyncWebCrawler with a depth parameter (e.g., depth=2 to capture sub-pages).
 3. **Consolidation:** The script merges all pages into a single antigravity_master_manual.md file.
 4. **Ingestion:** This file is moved to the agent's .agent/knowledge/ folder. Antigravity automatically indexes files in this directory for RAG (Retrieval-Augmented Generation), instantly making the agent an expert on the tool.²¹

4.2 The mcpdoc Standard: Standardized Knowledge Acquisition

For real-time documentation access without the overhead of crawling gigabytes of HTML, we utilize **mcpdoc**. This is an MCP server that acts as a "Translator," converting any llms.txt file on the web into a set of tools the agent can call.¹⁸

- **The Problem:** Many libraries do not yet have an llms.txt file.
- **The Self-Improving Solution:** The agent is configured with a fallback loop.
 1. Check for <https://<library>/llms.txt>.
 2. If found → Use mcpdoc to read it.
 3. If NOT found → Trigger crawl4ai to generate a local llms.txt from the HTML docs.
 4. Host this file locally and point mcpdoc to localhost.

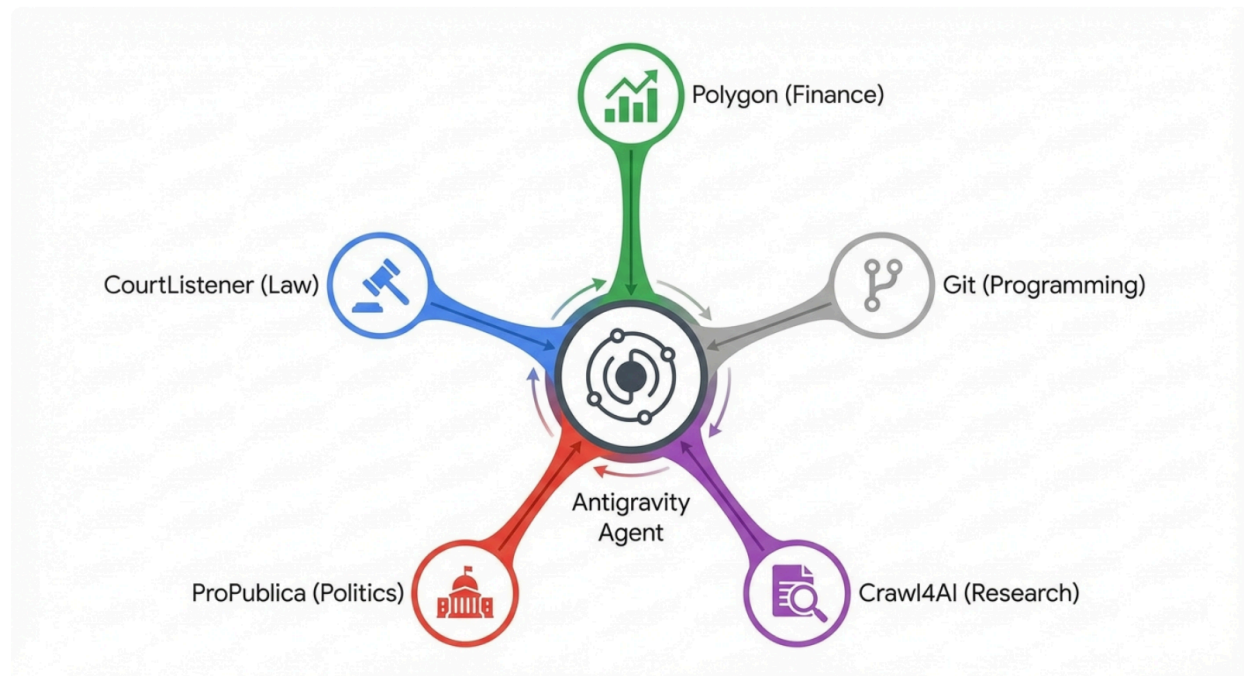
This creates a closed loop of knowledge acquisition. The agent effectively "writes the manual" for any tool it encounters that doesn't already have one.¹⁸

5. Part IV: Domain Mastery — The Ascending Ladder

The user defined a clear hierarchy of needs: **Programming < Research < Finance < Law <**

Politics. This is not merely a list of topics but a capability hierarchy. The agent uses Programming to build Research tools; uses Research tools to ingest Financial/Legal data; and synthesizes that data to understand Political macro-trends. We will now architect the specific "Skill Packs" for each domain, leveraging the **Model Context Protocol (MCP)** to plug these capabilities directly into the Antigravity agent.

The Specialized Agent's Toolbelt (MCP Ecology)



The agent achieves domain mastery by plugging into specialized MCP Servers. Each server acts as a sensory organ, providing structured data from that specific domain (e.g., Stock Prices, Court Opinions, Bills) to the central reasoning engine.

5.1 Level 1: Programming (The Base)

Goal: Self-modification, testing, and tool creation.

Tools: Antigravity Editor, Terminal, git-mcp.

The agent must be adept at **Recursive Code Generation**. It writes a script, executes it, reads the error from stderr, and rewrites the script. This is the "OODA Loop" of the coding agent.

- **Essential Skill:** git-mcp. This allows the agent to commit its own improvements. "I have improved my context compression script. Committing changes to branch self-improvement-v1".²⁴
- **Verification:** The agent must utilize the **Browser Preview** in Antigravity to visually verify its web apps (as seen in the Pomodoro example, where the agent iterates on the UI based on visual feedback).²⁶

- **Best Practice:** Integration of pre-commit hooks in the agent's workflow. The agent should be taught to run pytest before every commit. If tests fail, it enters a "Debugging Mode" loop rather than asking the human for help.

5.2 Level 2: Research (The Lens)

Goal: Finding "things" relevant to projects.

Tools: mcp-server-research, crawl4ai, Exa.ai API.

Research is not just searching; it is **synthesis**.

- **Deep Research Agent Pattern:** We configure a "Research Sub-Agent." This agent does not write code. It spawns, takes a query ("Best practices for Gemini CLI"), decomposes it into 5 search queries, visits 20 URLs, scrapes them with crawl4ai, and synthesizes a report.²⁷
- **Exa.ai Integration:** For high-precision retrieval, we replace generic Google Search with **Exa.ai** (formerly Metaphor), which is designed for LLMs. We wrap this in a custom MCP tool search_neural. Unlike keyword search, Exa understands semantic queries like "Find the API documentation for the Python wrapper of LegiScan" and returns the exact URL, which the agent then feeds to its crawler.²⁸

5.3 Level 3: Finance (The Market)

Goal: Understanding value flow, markets, and economic data.

Tools: mcp-server-polygon, mcp-server-alphavantage.

The agent requires real-time hard data. The user's interest here implies a need for an agent that can analyze the economic implications of its other tasks.

- **Polygon.io MCP:** This is the premier tool for this domain. It offers an official MCP server (mcp-server-polygon) that provides real-time access to stock, option, and crypto data.²⁹
- **Alpha Vantage MCP:** Excellent for "Fundamental Data" (Balance sheets, earnings reports, economic indicators like CPI/Inflation).³¹
- **The Financial Analyst Skill:** We create a finance_analyst skill. When invoked (e.g., "Backtest a mean-reversion strategy on NVDA"), it:
 1. Calls polygon.get_aggs for price history.
 2. Calls alpha_vantage.get_sentiment for news sentiment.
 3. Synthesizes a "Market State" object.
 4. Writes a Python backtesting script using pandas and vectorbt (a library the agent can install itself).
 5. Executes the backtest and reports the Sharpe Ratio.

Insight: The "Self-Improving" aspect here is **Backtesting**. The agent can "learn" the market physics through simulation, tweaking parameters and re-running the test until it finds an optimal strategy.

5.4 Level 4: Law (The Rules)

Goal: Compliance, precedent, and regulatory navigation.

Tools: CourtListener MCP, Free Law Project, US Legal Tools.

Legal research requires precision and citation. Hallucination is fatal here. The agent must rely on "Ground Truth" databases.

- **CourtListener MCP:** This is the definitive open-source tool connecting to the Free Law Project's database. It allows the agent to search millions of legal opinions.³³
- **US Legal Tools:** A comprehensive suite wrapping eCFR (Federal Regulations) and United States Code.³⁵

The Legal Agent Workflow:

When the user asks "Is this financial algorithm compliant with SEC Rule 10b-5?", the agent:

1. Identifies "SEC Rule 10b-5" as the key entity.
2. Uses `search_federal_register` (via US Legal Tools MCP) to pull the *exact text* of the rule.³⁶
3. Uses `search_court_opinions` (CourtListener) to find recent case law interpreting that rule.
4. Compares the user's Python code against the extracted legal logic.
5. **Output:** A compliance report citing specific paragraphs of the US Code and relevant case precedents.

5.5 Level 5: Politics (The Meta-Rules)

Goal: Legislation tracking, geopolitical risk, and macro-analysis.

Tools: Congress.gov API, ProPublica API, LegiScan.

This is the highest level of abstraction. Politics changes the *rules* of Law, which changes the *value* in Finance.

- **LegiScan API:** The most robust source for 50-state and federal legislative tracking.³⁷ Since there is no official MCP for LegiScan yet, the agent (using its Programming skill) will write a `legiscan_mcp.py` wrapper around the API.
- **Workflow:** The agent monitors specific keywords (e.g., "Artificial Intelligence Regulation," "Crypto Tax"). When a bill moves from "Introduced" to "Committee," the agent triggers an alert.
- **Cross-Domain Synthesis:** The "Self-Improving" agent connects the dots.
 - *Event:* "A new bill on 'AI Export Controls' has passed committee" (Politics).
 - *Impact:* "This modifies 'Title 15 US Code'" (Law).
 - *Implication:* "This will likely impact 'NVDA' stock price due to China exposure" (Finance).
 - *Action:* "I should update my 'Semiconductor Research' report" (Research).

The Unified Domain Intelligence Stack



The agent's intelligence stack is hierarchical. 'Politics' and 'Law' act as control layers that define the environment for 'Finance'. 'Research' and 'Programming' are the functional layers that allow the agent to observe and act within this environment. Arrows indicate the flow of causal impact.

6. Part V: The Synthesis — The Self-Improving Loop

The final requirement is that the agent "create an agential self-improving setup." This moves beyond executing tasks to *optimizing execution*.

6.1 The Meta-Reflection Protocol

To achieve this, the agent must be configured with a **Meta-Loop**.

1. **Reflection:** At the end of every task, the agent runs a "Reflection" step (often implemented as a separate prompt or agent).
 - "Did I have to look up documentation?"
 - "Did I encounter an error?"
 - "Did I write code that I later deleted?"
2. **Codification:** If the answer is yes, the agent creates a new SKILL.md or updates its MEMORY.md.³⁸
 - *Example:* If the agent struggled to format a matplotlib chart correctly, it saves a matplotlib_best_practices.md snippet to its knowledge base.

3. **Expansion:** Once a week (via a cron job or scheduled task), the agent runs its "Research Crawler" on its domain list to check for *new* documentation or *new* API versions.

6.2 The Bootstrap Prompt

To enable the "next Gemini 3 Pro instance" to create this setup, we must provide it with a **Bootstrap Protocol**. This is the prompt and instruction set you feed the model to kickstart the self-improvement cycle.

The "Genesis" Instruction:

"You are an Autonomous Architect Agent running in Google Antigravity. Your Goal is Recursive Self-Improvement.

Phase 1: Verify terminal access. If restricted, request 'admin.secureModeEnabled: false'.

Phase 2: Clone the 'crawl4ai' repository. Crawl '<https://antigravity.google/docs>' and compile 'antigravity_master_manual.md'. Index this manual.

Phase 3: Install MCP Servers for Polygon (Finance), CourtListener (Law), and LegiScan (Politics).

Phase 4: Establish the 'Context Flush' hook in settings.json.

Phase 5: Analyze your own error logs. Refine your 'SKILL.md' files. Commit changes to the 'self-improvement' branch."

By implementing this architecture, you do not just build a tool; you cultivate a digital organism that grows in competence alongside you. It respects the hierarchy of your interests, building a solid base of Programming and Research to support the high-level complexities of Finance, Law, and Politics.

Works cited

1. Build with Google Antigravity, our new agentic development platform, accessed January 25, 2026, <https://developers.googleblog.com/build-with-google-antigravity-our-new-agentic-development-platform/>
2. Getting Started with Google Antigravity, accessed January 25, 2026, <https://codelabs.developers.google.com/getting-started-google-antigravity>
3. Google Antigravity Documentation, accessed January 25, 2026, <https://antigravity.google/docs/home>
4. Gemini CLI Tutorial Series — Part 9: Understanding Context, Memory and Conversational Branching | by Romin Irani | Google Cloud - Medium, accessed January 25, 2026, <https://medium.com/google-cloud/gemini-cli-tutorial-series-part-9-understanding-g-context-memory-and-conversational-branching-095feb3e5a43>
5. Gemini CLI configuration, accessed January 25, 2026, <https://geminicli.com/docs/get-started/configuration/>
6. Gemini CLI Context Window Issues after reaching 50% of its capacity : r/GeminiAI - Reddit, accessed January 25, 2026, https://www.reddit.com/r/GeminiAI/comments/1netsfr/gemini_cli_context_window

- [_issues_after_reaching/](#)
7. Google's Advent of Agents — Day 6 | by AshJo | Dec, 2025 | Medium, accessed January 25, 2026, <https://medium.com/@ashwinjosh/googles-advent-of-agents-day-6-f9d161c91882>
 8. Quick Start - Crawl4AI Documentation (v0.7.x), accessed January 25, 2026, <https://docs.crawl4ai.com/core/quickstart/>
 9. Mastering Cross-Platform Deployment with Google Antigravity | by thenotorious runner | Dec, 2025 | Medium, accessed January 25, 2026, <https://medium.com/@thenotoriousrunner/mastering-cross-platform-deployment-with-google-antigravity-fae68d2eeb03>
 10. Antigravity Codes | 1,500+ MCP Servers, AI Rules & Workflows for Antigravity, Cursor & Windsurf, accessed January 25, 2026, <https://antigravity.codes/>
 11. How to Set Up and Use Google Antigravity - Codecademy, accessed January 25, 2026, <https://www.codecademy.com/article/how-to-set-up-and-use-google-antigravity>
 12. Agent Modes / Settings - Google Antigravity Documentation, accessed January 25, 2026, <https://antigravity.google/docs/agent-modes-settings>
 13. Allowlist / Denylist - Google Antigravity Documentation, accessed January 25, 2026, <https://antigravity.google/docs/allowlist-denylist>
 14. Tutorial : Getting Started with Google Antigravity Skills - Medium, accessed January 25, 2026, <https://medium.com/google-cloud/tutorial-getting-started-with-antigravity-skills-864041811e0d>
 15. Authoring Google Antigravity Skills, accessed January 25, 2026, <https://codelabs.developers.google.com/getting-started-with-antigravity-skills>
 16. Session Management | Gemini CLI, accessed January 25, 2026, <https://geminicli.com/docs/cli/session-management/>
 17. Gemini CLI supports several built-in commands to help you manage your session. - Reddit, accessed January 25, 2026, https://www.reddit.com/r/GeminiAI/comments/1na8c6v/gemini_cli_supports_several_built_in_commands_to/
 18. langchain-ai/mcpdoc: Expose llms-txt to IDEs for development - GitHub, accessed January 25, 2026, <https://github.com/langchain-ai/mcpdoc>
 19. Turn ANY Website into LLM Knowledge in SECONDS - YouTube, accessed January 25, 2026, https://www.youtube.com/watch?v=JWfNLF_g_V0
 20. Markdown Generation - Crawl4AI Documentation (v0.7.x), accessed January 25, 2026, <https://docs.crawl4ai.com/core/markdown-generation/>
 21. Documentation Crawler - Awesome MCP Servers, accessed January 25, 2026, https://mcpservers.org/servers/MCP-Mirror/alizdavoodi_MCPDocSearch
 22. docs-to-markdown - PyPI, accessed January 25, 2026, <https://pypi.org/project/docs-to-markdown/>
 23. The Modern Developer's Blueprint: Integrating Documentation-Aware Agents in Google Antigravity | by Ameya Suvarna | Google Cloud - Community | Jan, 2026 |

- Medium, accessed January 25, 2026,
<https://medium.com/google-cloud/the-modern-developers-blueprint-integrating-documentation-aware-agents-in-google-antigravity-016b87327867>
24. Example Servers - Model Context Protocol, accessed January 25, 2026,
<https://modelcontextprotocol.io/examples>
 25. Git MCP Server - playbooks, accessed January 25, 2026,
<https://playbooks.com/mcp/modelcontextprotocol/servers/git>
 26. Google Antigravity Tutorial for Beginners: Build Your First App (Step-by-Step), accessed January 25, 2026, <https://www.youtube.com/watch?v=-0lrz8G0PEE>
 27. Fundamentals of Building Autonomous LLM Agents This paper is based on a seminar technical report from the course Trends in Autonomous Agents: Advances in Architecture and Practice offered at TUM. - arXiv, accessed January 25, 2026, <https://arxiv.org/html/2510.09244v1>
 28. Exa | Web Search API, AI Search Engine, & Website Crawler, accessed January 25, 2026, <https://exa.ai/>
 29. Bridging AI and Blockchain: A Deep Dive into the Polygon MCP Server by Dablclub, accessed January 25, 2026,
<https://skywork.ai/skypage/en/ai-blockchain-polygon-mcp/1980477161358348288>
 30. Unveiling tddt's Stock Info (AKShare) MCP Server: A Deep Dive for AI Engineers, accessed January 25, 2026,
<https://skywork.ai/skypage/en/tddt-stock-info-ai-engineers/1978635030519992320>
 31. Best Stock Market APIs (Reviewed in 2025), accessed January 25, 2026,
https://www.alphavantage.co/best_stock_market_api_review/
 32. Hasan-Syed25/MCP-Server-AlphaVantage - GitHub, accessed January 25, 2026,
<https://github.com/Hasan-Syed25/MCP-Server-AlphaVantage>
 33. Travis-Prall/court-listener-mcp - GitHub, accessed January 25, 2026,
<https://github.com/Travis-Prall/court-listener-mcp>
 34. CourtListener Case Law Explorer | MCP Server - Smithery, accessed January 25, 2026, <https://smithery.ai/server/@khizar-anjum/courtlistener-mcp>
 35. beshkenadze/us-legal-tools - GitHub, accessed January 25, 2026,
<https://github.com/beshkenadze/us-legal-tools>
 36. TCoder920x/open-legal-compliance-mcp: Model Context Protocol server for legal compliance analysis using free/open government APIs - GitHub, accessed January 25, 2026, <https://github.com/TCoder920x/open-legal-compliance-mcp>
 37. LegiScan API, accessed January 25, 2026, <https://legiscan.com/legiscan>
 38. Let's Build a Self-Improving AI Agent That Learns From Your Feedback | by Nayeem Islam | Dec, 2025, accessed January 25, 2026,
<https://medium.com/@nomannayeem/lets-build-a-self-improving-ai-agent-that-learns-from-your-feedback-722d2ce9c2d9>